

Python爬虫

1.任务介绍

◆ 需求分析

爬取豆瓣电影Top250的基本信息，包括电影的名称、豆瓣评分、评价数、电影概况、电影链接等。

<https://movie.douban.com/top250>

豆瓣电影 Top 250

我没有看过的

- 

肖申克的救赎 / The Shawshank Redemption / 月黑高飞(港) / 刺激1995(台) [可播放]
导演: 弗兰克·德拉邦特 Frank Darabont 主演: 蒂姆·罗宾斯 Tim Robbins /...
1994 / 美国 / 犯罪 剧情
★★★★★ 9.6 1220094人评价
“希望让人自由。”
- 

霸王别姬 / 再见，我的妾 / Farewell My Concubine [可播放]
导演: 陈凯歌 Kaige Chen 主演: 张国荣 Leslie Cheung / 张丰毅 Fengyi Zha...
1993 / 中国大陆 香港 / 剧情 爱情 同性
★★★★★ 9.6 897570人评价
“风华绝代。”
- 

这个杀手不太冷 / Léon / 杀手莱昂 / 终极追杀令(台) [可播放]
导演: 吕克·贝松 Luc Besson 主演: 让·雷诺 Jean Reno / 娜塔莉·波特曼 ...
1994 / 法国 / 剧情 动作 犯罪
★★★★★ 9.4 1122631人评价
“怪事和小恶习不得不说的故事。”

2.爬虫初识

◆ 什么是爬虫

网络爬虫，是一种按照一定规则，自动抓取互联网信息的程序或者脚本。由于互联网数据的多样性和资源的有限性，根据用户需求定向抓取相关网页并分析已成为如今主流的爬取策略。

◆ 爬虫可以做什么

你可以爬取妹子的图片，爬取自己想看的视频等等，只要你能通过浏览器访问的数据都可以通过爬虫获取。

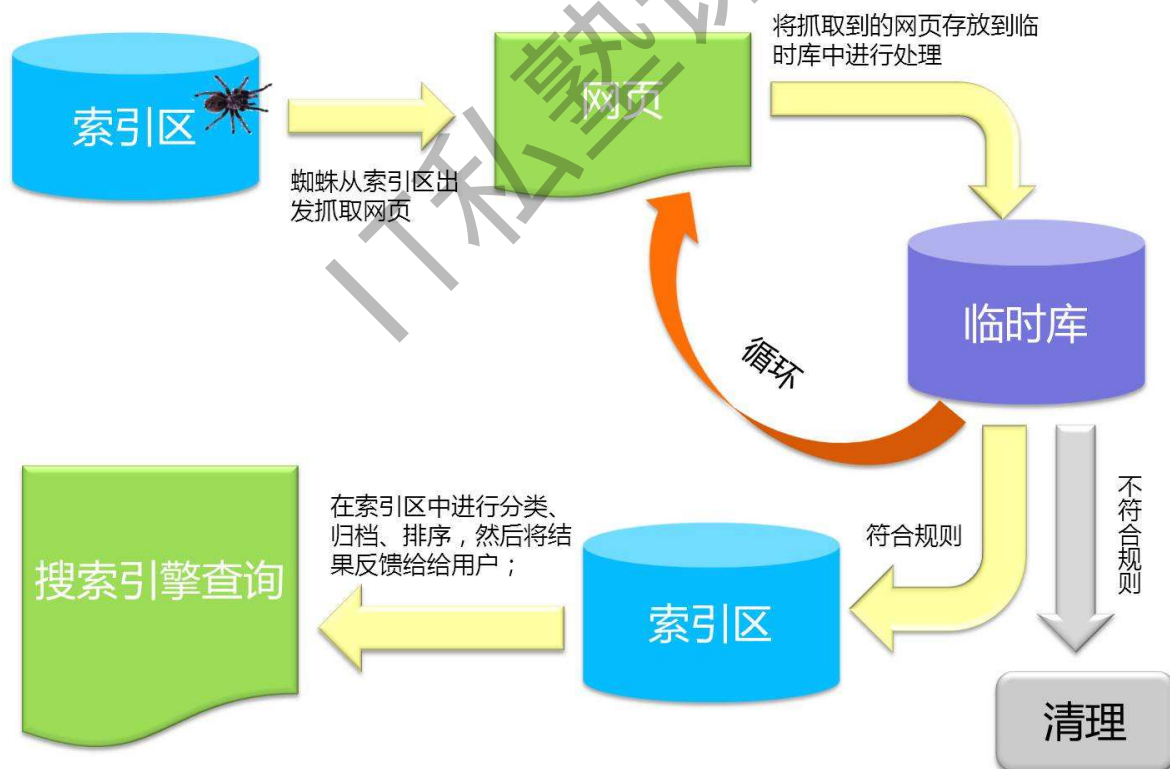
◆ 爬虫的本质是什么

模拟浏览器打开网页，获取网页中我们想要的那部分数据。

蜘蛛的爬行原理



搜索引擎原理图



3.基本流程

◆ 准备工作

通过浏览器查看分析目标网页，学习编程基础规范。

◆ 获取数据

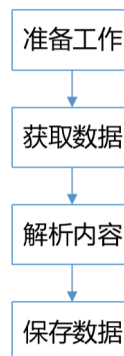
通过HTTP库向目标站点发起请求，请求可以包含额外的header等信息，如果服务器能正常响应，会得到一个Response，便是所要获取的页面内容。

◆ 解析内容

得到的内容可能是HTML、json等格式，可以用[页面解析库](#)、[正则表达式](#)等进行解析。

◆ 保存数据

保存形式多样，可以存为文本，也可以保存到数据库，或者保存特定格式的文件。



3.1 准备工作

<https://movie.douban.com/top250?start=0>

豆瓣电影 Top 250



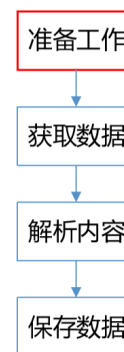
<https://movie.douban.com/top250?start=25>

豆瓣电影 Top 250



✓ URL分析

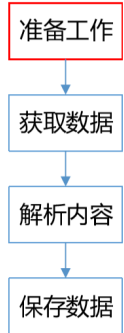
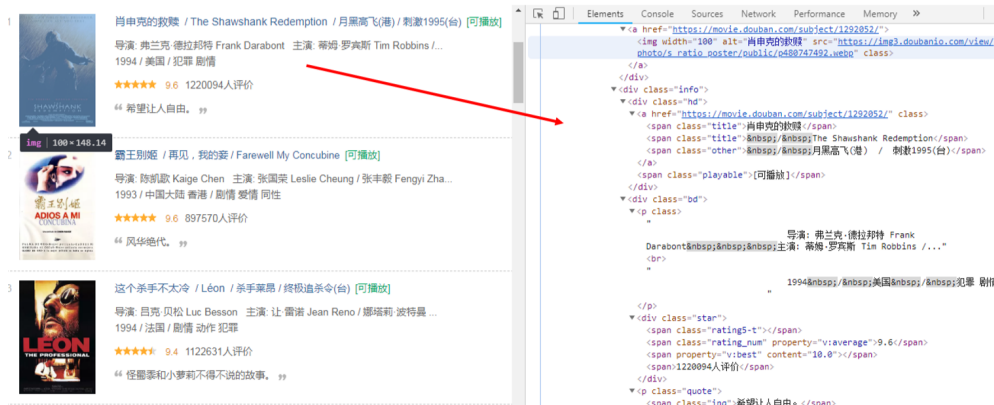
- ✓ 页面包括250条电影数据，分10页，每页25条
- ✓ 每页的URL的不同之处：最后的数值 = (页数 - 1) * 25



提示：本节课程需要HTML、CSS、JS的基础知识，同学可以自行复习一下。

3.1.1 分析页面

◆ 借助Chrome开发者工具（F12）来分析网页，在Elements下找到需要的数据位置



3.1.2 编码规范

◆ 一般Python程序第一行需要加入

```
# -*- coding: utf-8 -*- 或者 # coding=utf-8
```

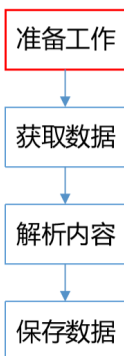
这样可以在代码中包含中文

◆ 在Python中，使用函数实现单一功能或相关联功能的代码段，可以提高可读性和代码重复利用率，函数代码块以def关键词开头，后接空格、函数标识符名称、圆括号()、冒号:，括号中可以传入参数，函数段缩进（Tab或四个空格，只能任选一种），return用于结束函数，可以返回一个值，也可以不带任何表达式（表示返回None）

◆ Python文件中可以加入main函数用于测试程序

```
if __name__ == "__main__":
```

◆ Python使用#添加注释，说明代码（段）的作用



3.1.3 引入模块

◆ 模块（module）：用来从逻辑上组织Python代码（变量、函数、类），本质就是py文件，提高代码的可维护性。Python使用import来导入模块，如

```
import sys
```

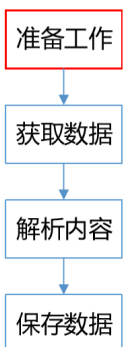
```
import sys
from bs4 import BeautifulSoup
import re
import urllib
import xlwt
```

```
def main():
    print("开始爬取.....")
    baseurl='https://movie.douban.com/top250?start='
    datalist=getData(baseurl)
    savapath='u:/home/aistudio/data/豆瓣电影Top250.xls'
    saveData(datalist, savapath)
```

```
main()
print("爬取完成，请查看.xls文件")
```

✓ 主流程

✓ 获取数据、解析内容、保存数据



模块 module：一般情况下，是一个以.py为后缀的文件。

module 可看作一个工具类，可共用或者隐藏代码细节，将相关代码放置在一个module以便让代码更好用、易懂，让coder重点放在高层逻辑上。

module能定义函数、类、变量，也能包含可执行的代码。module来源有3种：

- ①Python内置的模块（标准库）；
- ②第三方模块；
- ③自定义模块。

包 package：为避免模块名冲突，Python引入了按目录组织模块的方法，称之为包（package）。包是含有Python模块的文件夹。

3.2 获取数据

◆ Python一般使用urllib2库获取页面

```
for i in range(0,10):  
    url=baseurl+str(i*25)  
    html=askURL(url)
```

```
#得到页面全部内容  
def askURL(url):  
    request = urllib.request.Request(url)#发送请求  
    try:  
        response = urllib.request.urlopen(request)#获得响应  
        html= response.read()#读取网页内容  
        print (html)  
    except urllib.error.URLError as e:  
        if hasattr(e, "code"):  
            print (e.code)  
        if hasattr(e, "reason"):  
            print (e.reason)  
    return html
```

✓ 获取页面数据

✓ 对每一个页面，调用askURL函数获取页面内容

✓ 定义一个获取页面的函数askURL，传入一个url参数，表示网址，如

<https://movie.douban.com/top250?start=0>

✓ urllib2.Request生成请求；
urllib2.urlopen发送请求获取响应；read获取页面内容

✓ 在访问页面时经常会出现错误，为了程序正常运行，加入异常捕获try...except...语句

准备工作

获取数据

解析内容

保存数据

补充：urllib模块

最基本的请求

是python内置的一个http请求库，不需要额外的安装。只需要关注请求的链接，参数，提供了强大的解析。

urllib.request 请求模块
urllib.error 异常处理模块
urllib.parse 解析模块

- 用法讲解
简单的一个get请求

```
import urllib.request
reponse = urllib.request.urlopen('http://www.baidu.com')
print(reponse.read().decode('utf-8'))
```

简单的一个post请求

```
import urllib.parse
import urllib.request
data = bytes(urllib.parse.urlencode({'hello': 'world'}), encoding='utf-8')
reponse = urllib.request.urlopen('http://httpbin.org/post', data=data)
print(reponse.read())
```

超时处理

```
import urllib.request
response = urllib.request.urlopen('http://httpbin.org/get', timeout=1)
print(response.read())
```

```
import urllib.request
import socket
import urllib.error
try:
    response = urllib.request.urlopen('http://httpbin.org/get', timeout=0.01)
except urllib.error.URLError as e:
    if isinstance(e.reason, socket.timeout): #判断错误原因
        print('time out!')
```

打印出响应类型，状态码，响应头

```
import urllib.request
response=urllib.request.urlopen('http://www.baidu.com')
print(type(response))
```

```
import urllib.request
response = urllib.request.urlopen('http://www.baidu.com')
print(response.status) # 状态码 判断请求是否成功
print(response.getheaders()) # 响应头 得到的一个元组组成的列表
print(response.getheader('Server')) #得到特定的响应头
print(response.read().decode('utf-8')) #获取响应体的内容，字节流的数据，需要转成utf-8格式
```

由于使用urlopen无法传入参数，我们需要解决这个问题
我们需要声明一个request对象，通过这个对象来添加参数

```
import urllib.request
request = urllib.request.Request('https://python.org') #由于urlopen无法传参数，声明一个Request对象
response = urllib.request.urlopen(request)
print(response.read().decode('utf-8'))
```

我们还可以分别创建字符串、字典等等来带入到request对象里面

```
from urllib import request, parse
url='http://httpbin.org/post'
headers={
    'user-agent': 'Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.98 Safari/537.36',
    'Host': 'httpbin.org'
}
dict={
    'name': 'jay'
}
data = bytes(parse.urlencode(dict), encoding='utf-8')
req=request.Request(url=url, data=data, headers=headers, method='POST')
response=request.urlopen(req)
print(response.read().decode('utf-8'))
```

我们还可以通过addheaders方法不断的向原始的requests对象里不断添加

```
from urllib import request, parse
url = 'http://httpbin.org/post'
dict = {
    'name': 'cq'
}
data=bytes(parse.urlencode(dict), encoding='utf-8')
req = request.Request(url=url, data=data, method='POST')
req.add_header('user-agent', 'Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.98 Safari/537.36')
response=request.urlopen(req)
print(response.read().decode('utf-8'))
```

打印出信息cookies

下面这段程序获取response后声明的cookie对象会被自动赋值

```
import http.cookiejar,urllib.request
cookie = http.cookiejar.CookieJar()
handler=urllib.request.HTTPCookieProcessor(cookie)
opener=urllib.request.build_opener(handler)
response=opener.open('http://www.baidu.com') #获取response后cookie会被自动赋值
for item in cookie:
    print(item.name+'='+item.value)
```

保存cookie文件,两种格式

```
import http.cookiejar,urllib.request
filename='cookie.txt'
cookie = http.cookiejar.MozillaCookieJar(filename)
handler=urllib.request.HTTPCookieProcessor(cookie)
opener=urllib.request.build_opener(handler)
response=opener.open('http://www.baidu.com') #获取response后cookie会被自动赋值
cookie.save(ignore_discard=True,ignore_expires=True) #保存cookie.txt文件
```

```
import http.cookiejar,urllib.request
filename='cookie2.txt'
cookie = http.cookiejar.LWPCookieJar(filename)
handler=urllib.request.HTTPCookieProcessor(cookie)
opener=urllib.request.build_opener(handler)
response=opener.open('http://www.baidu.com') #获取response后cookie会被自动赋值
cookie.save(ignore_discard=True,ignore_expires=True) #保存cookie.txt文件
```

用文本文件的形式维持登录状态

```
import http.cookiejar,urllib.request
cookie = http.cookiejar.MozillaCookieJar()
cookie.load('cookie.txt',ignore_discard=True,ignore_expires=True)
handler=urllib.request.HTTPCookieProcessor(cookie)
opener=urllib.request.build_opener(handler)
response=opener.open('http://www.baidu.com')
print(response.read().decode('utf-8'))
```

关于异常处理部分,需要了解有httperror和urlerror两种,父类与子类的关系。


```

#父类, 只有一个reason
from urllib import request,error
try:
    response = request.urlopen('http://www.bai.com/index.html')
except error.URLError as e:
    print(e.reason)

#子类, 有更多的属性
from urllib import request,error
try:
    response = request.urlopen('http://abc.123/index.html')
except error.HTTPError as e:
    print(e.reason,e.code,e.headers,sep='\n')

```

解析,将一个url解析

```

from urllib.parse import urlparse
result = urlparse('http://www.baidu.com/index.html;user?id=5#comment')
print(result)#协议内容、路径、参数
print(type(result))

from urllib.parse import urlparse
result = urlparse('www.baidu.com/index.html;user?id=5#comment',scheme='https')
print(result)

from urllib.parse import urlparse
result = urlparse('http://www.baidu.com/index.html;user?id=5#comment',scheme='https')
print(result)

from urllib.parse import urlparse
result = urlparse('http://www.baidu.com/index.html;user?id=5#comment',allow_fragments=False) #会被拼接
print(result)

from urllib.parse import urlparse
result =
urlparse('http://www.baidu.com/index.html#comment',allow_fragments=False) #会被
拼接到path没有query
print(result)

```

url拼接

```

from urllib.parse import urlunparse
data=['http','www.baidu.com','index.html','user','a=6','comment']
print(urlunparse(data))

```

```
from urllib.parse import urljoin
#拼接两个url
#截图，以后面的为基准，有留下，没有拼接

print(urljoin('http://www.baidu.com', 'HAA.HTML'))
print(urljoin('https://www.baidu.com', 'https://www.baidu.com/index.html;question=2'))
```

```
#字典方式直接转换成url参数
from urllib.parse import urlencode
params = {
    'name': 'germey',
    'age': '122'
}
base_url='http://www.baidu.com?'
url=base_url+urlencode(params)
print(url)
```

更多细节可以参考: <https://www.cnblogs.com/qikeyishu/p/10748497.html>

3.3 解析内容

◆ 对爬取的html文件进行解析

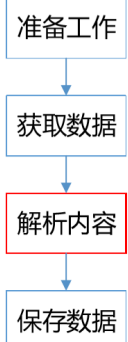
```
soup = BeautifulSoup(html, "html.parser")
for item in soup.find_all('div', class_='item'): #找到每一个影片项
    data=[]
    item=str(item) #转换成字符串
    # 影片详情链接
    link=re.findall(findLink, item)[0]
    data.append(link) #添加详情链接

    imgSrc=re.findall(findImgSrc, item)[0]
    data.append(imgSrc) #添加图片链接

    titles=re.findall(findTitle, item)
    #片名可能只有一个中文名，没有外国名
    if len(titles)==2:
        ctitle=titles[0]
        data.append(ctitle) #添加中文片名
        otitle=titles[1].replace("/","") #去掉无关符号
```

✓ 解析页面内容

- ✓ 使用BeautifulSoup定位特定的标签位置
- ✓ 使用正则表达式找到具体的内容



3.3.1 标签解析

◆ Beautiful Soup

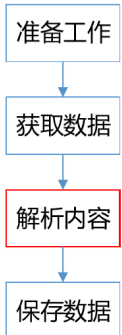
Beautiful Soup是一个库，提供一些简单的、python式的用来处理导航、搜索、修改分析树等功能，通过解析文档为用户提供需要抓取的数据。我们需要的每个电影都在一个<div>的标签中，且每个div标签都有一个属性class= "item" 。

```
soup = BeautifulSoup(html, "html.parser")
for item in soup.find_all('div', class_='item'): #找到每一个影片项
```

创建BeautifulSoup 对象，html为页面内容，html.parser是一种页面解析器

找到能够完整提取出一个影片内容的项，即页面中所有样式是item类的div

```
<li>
  <div class="item"></div>
</li>
<li>
  <div class="item"></div>
</li>
<li>
  <div class="item"></div>
</li>
<li>
  <div class="item"></div>
</li>
<li>
  <div class="item"></div>
</li>
</li>
</li>
```



补充：BeautifulSoup模块

参考：<http://www.jsphp.net/python/show-24-214-1.html>

1.BeautifulSoup4简介

BeautifulSoup4和 lxml 一样，Beautiful Soup 也是一个HTML/XML的解析器，主要的功能也是如何解析和提取 HTML/XML 数据。

BeautifulSoup支持Python标准库中的HTML解析器,还支持一些第三方的解析器，如果我们不安装它，则 Python 会使用 Python默认的解析器，lxml 解析器更加强大，速度更快，推荐使用lxml 解析器。

Beautiful Soup自动将输入文档转换为Unicode编码，输出文档转换为utf-8编码。你不需要考虑编码方式，除非文档没有指定一个编码方式，这时，Beautiful Soup就不能自动识别编码方式了。然后，你仅仅需要说明一下原始编码方式就可以了。

2.BeautifulSoup4主要解析器，以及优缺点：

解析器	使用方法	优势	劣势
Python标准库	<code>BeautifulSoup(markup, "html.parser")</code>	Python的内置标准库 执行速度适中 文档容错能力强	Python 2.7.3 or 3.2.2)前 的版本中文档容错能力差
lxml HTML 解析器	<code>BeautifulSoup(markup, "lxml")</code>	速度快 文档容错能力强	需要安装C语言库
lxml XML 解析器	<code>BeautifulSoup(markup, ["lxml-xml"])</code> <code>BeautifulSoup(markup, "xml")</code>	速度快 唯一支持XML的解析器	需要安装C语言库
html5lib	<code>BeautifulSoup(markup, "html5lib")</code>	最好的容错性 以浏览器的方式解析文档 生成HTML5格式的文档	速度慢 不依赖外部扩展

3.BeautifulSoup4简单使用

假设有这样一个Html，具体内容如下：

```
<!DOCTYPE html>
<html>
<head>
  <meta content="text/html; charset=utf-8" http-equiv="content-type" />
  <meta content="IE=Edge" http-equiv="X-UA-Compatible" />
  <meta content="always" name="referrer" />
  <link
href="https://ss1.bdstatic.com/5eN1bjq8AAUYm2zgoY3K/r/www/cache/bdorz/baidu.min.
css" rel="stylesheet" type="text/css" />
  <title>百度一下，你就知道 </title>
</head>
<body link="#0000cc">
  <div id="wrapper">
    <div id="head">
      <div class="head_wrapper">
        <div id="u1">
          <a class="mnav" href="http://news.baidu.com" name="tj_trnews">新闻
</a>
          <a class="mnav" href="https://www.hao123.com"
name="tj_trhao123">hao123 </a>
          <a class="mnav" href="http://map.baidu.com" name="tj_trmap">地图 </a>
          <a class="mnav" href="http://v.baidu.com" name="tj_trvideo">视频 </a>
          <a class="mnav" href="http://tieba.baidu.com" name="tj_trtieba">贴吧
</a>
          <a class="bri" href="//www.baidu.com/more/" name="tj_briicon"
style="display: block;">更多产品 </a>
        </div>
      </div>
    </div>
  </div>
</body>
</html>
```

创建beautifulsoup4对象：

```
from bs4 import BeautifulSoup
file = open('./aa.html', 'rb')
html = file.read()
bs = BeautifulSoup(html, "html.parser") # 缩进格式

print(bs.prettify()) # 获取title标签的所有内容
print(bs.title) # 获取title标签的名称
print(bs.title.name) # 获取title标签的文本内容
print(bs.title.string) # 获取head标签的所有内容
print(bs.head) # 获取第一个div标签中的所有内容
print(bs.div) # 获取第一个div标签的id的值
print(bs.div["id"]) # 获取第一个a标签中的所有内容
print(bs.a) # 获取所有的a标签中的所有内容
print(bs.find_all("a")) # 获取id="u1"
```

```
print(bs.find(id="u1")) # 获取所有的a标签，并遍历打印a标签中的href的值

for item in bs.find_all("a"):
    print(item.get("href")) # 获取所有的a标签，并遍历打印a标签的文本值

for item in bs.find_all("a"):
    print(item.get_text())
```

4.BeautifulSoup4四大对象种类

BeautifulSoup4将复杂HTML文档转换成一个复杂的树形结构,每个节点都是Python对象,所有对象可以归纳为4种:

- Tag
- NavigableString
- BeautifulSoup
- Comment

(1) Tag

Tag通俗点讲就是HTML中的一个标签，例如：

```
from bs4 import BeautifulSoup
file = open('./aa.html', 'rb')
html = file.read()
bs = BeautifulSoup(html, "html.parser")
# 获取title标签的所有内容
print(bs.title)
# 获取head标签的所有内容
print(bs.head)
# 获取第一个a标签的所有内容
print(bs.a)
# 类型
print(type(bs.a))
```

我们可以利用 soup 加标签名轻松地获取这些标签的内容，这些对象的类型是bs4.element.Tag。但是注意，它查找的是在所有内容中的第一个符合要求的标签。

对于 Tag，它有两个重要的属性，是 name 和 attrs：

```
from bs4 import BeautifulSoup
file = open('./aa.html', 'rb')
html = file.read()
bs = BeautifulSoup(html, "html.parser")
# [document] #bs 对象本身比较特殊，它的 name 即为 [document]
print(bs.name)
# head #对于其他内部标签，输出的值便为标签本身的名称
print(bs.head.name)
# 在这里，我们把 a 标签的所有属性打印输出出来了，得到的类型是一个字典。
print(bs.a.attrs)
#还可以利用get方法，传入属性的名称，二者是等价的
```

```

print(bs.a['class']) # 等价 bs.a.get('class')
# 可以对这些属性和内容等等进行修改
bs.a['class'] = "newClass"
print(bs.a)
# 还可以对这个属性进行删除
del bs.a['class']
print(bs.a)

```

4.2、NavigableString

既然我们已经得到了标签的内容，那么问题来了，我们要想获取标签内部的文字怎么办呢？很简单，用 `.string` 即可，例如

```

from bs4 import BeautifulSoup
file = open('./aa.html', 'rb')
html = file.read()
bs = BeautifulSoup(html, "html.parser")

print(bs.title.string)
print(type(bs.title.string))

```

4.3、BeautifulSoup

BeautifulSoup对象表示的是一个文档的内容。大部分时候，可以把它当作 Tag 对象，是一个特殊的 Tag，我们可以分别获取它的类型，名称，以及属性，例如：

```

from bs4 import BeautifulSoup
file = open('./aa.html', 'rb')
html = file.read()

bs = BeautifulSoup(html, "html.parser")
print(type(bs.name))
print(bs.name)
print(bs.attrs)

```

4.4、Comment

Comment 对象是一个特殊类型的 NavigableString 对象，其输出的内容不包括注释符号。

```

from bs4 import BeautifulSoup
file = open('./aa.html', 'rb')
html = file.read()
bs = BeautifulSoup(html, "html.parser")
print(bs.a)
# 此时不能出现空格和换行符，a标签如下：
# <a class="mnav" href="http://news.baidu.com" name="tj_trnews"><!--新闻--></a>
print(bs.a.string) # 新闻
print(type(bs.a.string)) # <class 'bs4.element.Comment'>

```

五、遍历文档树

5.1 .contents: 获取Tag的所有子节点, 返回一个list

```
# tag的.content 属性可以将tag的子节点以列表的方式输出
print(bs.head.contents)
# 用列表索引来获取它的某一个元素
print(bs.head.contents[1])
```

5.2 .children: 获取Tag的所有子节点, 返回一个生成器

```
for child in bs.body.children:
    print(child)
```

5.3、.descendants: 获取Tag的所有子孙节点

5.4、.strings: 如果Tag包含多个字符串, 即在子孙节点中有内容, 可以用此获取, 而后进行遍历

5.5、.stripped_strings: 与strings用法一致, 只不过可以去除掉那些多余的空白内容

5.6、.parent: 获取Tag的父节点

5.7、.parents: 递归得到父辈元素的所有节点, 返回一个生成器

5.8、.previous_sibling: 获取当前Tag的上一个节点, 属性通常是字符串或空白, 真实结果是当前标签与上一个标签之间的顿号和换行符

5.9、.next_sibling: 获取当前Tag的下一个节点, 属性通常是字符串或空白, 真实结果是当前标签与下一个标签之间的顿号与换行符

5.10、.previous_siblings: 获取当前Tag的上面所有的兄弟节点, 返回一个生成器

5.11、.next_siblings: 获取当前Tag的下面所有的兄弟节点, 返回一个生成器

5.12、.previous_element: 获取解析过程中上一个被解析的对象(字符串或tag), 可能与previous_sibling相同, 但通常是不一样的

5.13、.next_element: 获取解析过程中下一个被解析的对象(字符串或tag), 可能与next_sibling相同, 但通常是不一样的

5.14、.previous_elements: 返回一个生成器, 可以向前访问文档的解析内容

5.15、.next_elements: 返回一个生成器, 可以向后访问文档的解析内容

5.16、.has_attr: 判断Tag是否包含属性

六、搜索文档树

6.1、find_all(name, attrs, recursive, text, **kwargs)

在上面的例子中我们简单介绍了find_all的使用，接下来介绍一下find_all的更多用法-过滤器。这些过滤器贯穿整个搜索API，过滤器可以被用在tag的name中，节点的属性等。

(1) name参数:

字符串过滤：会查找与字符串完全匹配的内容

```
a_list = bs.find_all("a")
print(a_list)
```

正则表达式过滤：如果传入的是正则表达式，那么BeautifulSoup4会通过search()来匹配内容

```
from bs4 import BeautifulSoup
import re
file = open('./aa.html', 'rb')
html = file.read()
bs = BeautifulSoup(html, "html.parser")
t_list = bs.find_all(re.compile("a"))
for item in t_list:
    print(item)
```

列表：如果传入一个列表，BeautifulSoup4将会与列表中的任一元素匹配到的节点返回

```
t_list = bs.find_all(["meta", "link"])
for item in t_list:
    print(item)
```

方法：传入一个方法，根据方法来匹配

```
from bs4 import BeautifulSoup
file = open('./aa.html', 'rb')
html = file.read()
bs = BeautifulSoup(html, "html.parser")
def name_is_exists(tag):
    return tag.has_attr("name")
t_list = bs.find_all(name_is_exists)
for item in t_list:
    print(item)
```

(2) kwargs参数:

```
from bs4 import BeautifulSoup
import re
file = open('./aa.html', 'rb')
html = file.read()
bs = BeautifulSoup(html, "html.parser")
# 查询id=head的Tag
t_list = bs.find_all(id="head")
```



```

print(t_list)
# 查询href属性包含ssl.bdstatic.com的Tag
t_list = bs.find_all(href=re.compile("http://news.baidu.com"))
print(t_list)
# 查询所有包含class的Tag(注意: class在Python中属于关键字, 所以加_以示区别)
t_list = bs.find_all(class_=True)
for item in t_list:
    print(item)

```

(3) attrs参数:

并不是所有的属性都可以使用上面这种方式进行搜索, 比如HTML的data-*属性:

```
t_list = bs.find_all(data-foo="value")
```

如果执行这段代码, 将会报错。我们可以使用attrs参数, 定义一个字典来搜索包含特殊属性的tag:

```

t_list = bs.find_all(attrs={"data-foo": "value"})
for item in t_list:
    print(item)

```

(4) text参数:

通过text参数可以搜索文档中的字符串内容, 与name参数的可选值一样, text参数接受 字符串, 正则表达式, 列表

```

from bs4 import BeautifulSoup
import re
file = open('./aa.html', 'rb')
html = file.read()
bs = BeautifulSoup(html, "html.parser")
t_list = bs.find_all(attrs={"data-foo": "value"})
for item in t_list:
    print(item)
t_list = bs.find_all(text="hao123")
for item in t_list:
    print(item)
t_list = bs.find_all(text=["hao123", "地图", "贴吧"])
for item in t_list:
    print(item)
t_list = bs.find_all(text=re.compile("\d"))
for item in t_list:
    print(item)

```

当我们搜索text中的一些特殊属性时, 同样也可以传入一个方法来达到我们的目的:

```
def length_is_two(text):
    return text and len(text) == 2
t_list = bs.find_all(text=length_is_two)
for item in t_list:
    print(item)
```

(5) limit参数:

可以传入一个limit参数来限制返回的数量，当搜索出的数据量为5，而设置了limit=2时，此时只会返回前2个数据

```
from bs4 import BeautifulSoup
import re
file = open('./aa.html', 'rb')
html = file.read()
bs = BeautifulSoup(html, "html.parser")
t_list = bs.find_all("a", limit=2)
for item in t_list:
    print(item)
```

find_all除了上面一些常规的写法，还可以对其进行一些简写：

```
# 两者是相等的
# t_list = bs.find_all("a") => t_list = bs("a")
t_list = bs("a") # 两者是相等的
# t_list = bs.a.find_all(text="新闻") => t_list = bs.a(text="新闻")
t_list = bs.a(text="新闻")
```

6.2、find()

find()将返回符合条件的第一个Tag，有时我们只需要或一个Tag时，我们就可以用到find()方法了。当然了，也可以使用find_all()方法，传入一个limit=1，然后再取出第一个值也是可以的，不过未免繁琐。

```
from bs4 import BeautifulSoup
import re
file = open('./aa.html', 'rb')
html = file.read()
bs = BeautifulSoup(html, "html.parser")
# 返回只有一个结果的列表
t_list = bs.find_all("title", limit=1)
print(t_list)
# 返回唯一值
t = bs.find("title")
print(t)
# 如果没有找到，则返回None
t = bs.find("abc") print(t)
```

从结果可以看出find_all，尽管传入了limit=1，但是返回值仍然为一个列表，当我们只需要取一个值时，远不如find方法方便。但

是如果未搜索到值时，将返回一个None

在上面介绍BeautifulSoup4的时候，我们知道可以通过bs.div来获取第一个div标签，如果我们需要获取第一个div下的第一个div，

我们可以这样：

```
t = bs.div.div
# 等价于
t = bs.find("div").find("div")
```

7.CSS选择器

BeautifulSoup支持发部分的CSS选择器，在Tag获取BeautifulSoup对象的.select()方法中传入字符串参数，即可使用CSS选择器的语法找到Tag:

7.1、通过标签名查找

```
print(bs.select('title'))
print(bs.select('a'))
```

7.2、通过类名查找

```
print(bs.select('.mnav'))
```

7.3、通过id查找

```
print(bs.select('#u1'))
```

7.4、组合查找

```
print(bs.select('div .bri'))
```

7.5、属性查找

```
print(bs.select('a[class="bri"]'))
print(bs.select('a[href="http://tieba.baidu.com"]'))
```

7.6、直接子标签查找

```
t_list = bs.select("head > title")
print(t_list)
```

7.7、兄弟节点标签查找

```
t_list = bs.select(".mnav ~ .bri")
print(t_list)
```

7.8、获取内容

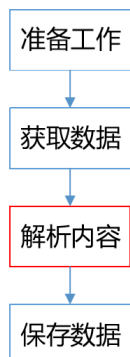
```
t_list = bs.select("title")
print(bs.select('title')[0].get_text())
```

3.3.2 正则提取

◆ 正则表达式

正则表达式，通常被用来检索、替换那些符合某个模式（规则）的文本。正则表达式是对字符串操作的一种逻辑公式，就是用事先定义好的一些特定字符及这些特定字符的组合，组成一个“规则字符串”，这个“规则字符串”用来表达对字符串的一种过滤逻辑。Python中使用re模块操作正则表达式。

```
findLink=re.compile(r'<a href="(.*?)>' )#抓到影片详情链接
findImgSrc=re.compile(r'(.*?)</span>' )#抓到片名
#抓到评分
findRating=re.compile(r'<span class="rating_num" property="v:average">(.*?)</span>' )
#抓到评价人数
findJudge=re.compile(r'<span>(\d*)人评价</span>' )
#抓到概况
findInq=re.compile(r'<span class="inq">(.*?)</span>' )
#抓到影片相关内容：导演，主演，年份，地区，类别
findBd=re.compile(r'<p class="">(.*?)</p>',re.S)
```



补充：re模块

正则表达式的常用操作符(1)

操作符	说明	实例
.	表示任何单个字符	
[]	字符集，对单个字符给出取值范围	[abc]表示a、b、c，[a-z]表示a到z单个字符
[^]	非字符集，对单个字符给出排除范围	[^abc]表示非a或b或c的单个字符
*	前一个字符0次或无限次扩展	abc* 表示 ab、abc、abcc、abccc等
+	前一个字符1次或无限次扩展	abc+ 表示 abc、abcc、abccc等
?	前一个字符0次或1次扩展	abc? 表示 ab、abc
	左右表达式任意一个	abc def 表示 abc、def

正则表达式的常用操作符(2)

操作符	说明	实例
{m}	扩展前一个字符m次	ab{2}c表示abbc
{m,n}	扩展前一个字符m至n次（含n）	ab{1,2}c表示abc、abbc
^	匹配字符串开头	^abc表示abc且在一个字符串的开头
\$	匹配字符串结尾	abc\$表示abc且在一个字符串的结尾
()	分组标记，内部只能使用 操作符	(abc)表示abc，(abc def)表示abc、def
\d	数字，等价于[0-9]	
\w	单词字符，等价于[A-Za-z0-9_]	

Re库主要功能函数

函数	说明
re.search()	在一个字符串中搜索匹配正则表达式的第一个位置，返回match对象
re.match()	从一个字符串的开始位置起匹配正则表达式，返回match对象
re.findall()	搜索字符串，以列表类型返回全部能匹配的子串
re.split()	将一个字符串按照正则表达式匹配结果进行分割，返回列表类型
re.finditer()	搜索字符串，返回一个匹配结果的迭代类型，每个迭代元素是match对象
re.sub()	在一个字符串中替换所有匹配正则表达式的子串，返回替换后的字符串

正则表达式可以包含一些可选标志修饰符来控制匹配的模式。修饰符被指定为一个可选的标志。多个标志可以通过按位 OR(|) 它们来指定。如 re.I | re.M 被设置成 I 和 M 标志：

修饰符	描述
re.I	使匹配对大小写不敏感
re.L	做本地化识别 (locale-aware) 匹配
re.M	多行匹配，影响 ^ 和 \$
re.S	使 . 匹配包括换行在内的所有字符
re.U	根据Unicode字符集解析字符。这个标志影响 \w, \W, \b, \B.
re.X	该标志通过给予你更灵活的格式以便你将正则表达式写得更易于理解。

###

re模块下的函数

compile(pattern): 创建模式对象

```

import re
pat=re.compile("A")
#m=pat.search("CBA")
m=pat.search("ABC")
#等价于 re.search( A , CBA )
print(m)

#<re.Match object; span=(2, 3), match='A'> 表示匹配到了

m=pat.search("CBD")
print(m)          #None 表示没匹配到

```

search(pattern,string): 在字符串中寻找模式

```

import re
m = re.search("asd" , "ASDasd" )
print(m)
# <_sre.SRE_Match object at 0xb72cd6e8> #匹配到了, 返回MatchObject (True)
m = re.search("asd" , "ASDASD" )
print(m)                                #没有匹配到, 返回None (False)

```

match(pattern,string): 在字符串开始处匹配模式

```

# 等价于
pat=re.compile( "a" )
print(pat.match( "Aasd" ))
#输出None

print(pat.match("aASD" ))
#输出 <_sre.SRE_Match object at 0xb72cd6e8>

# 上面的函数返回都可以在if条件语句中进行判断:

if pat.search("asd"):
    print ("OK")      #OK          #找到返回

if re.search("a","ASD" ):
    print ("OK")      #没有找到

```

split(pattern,string): 根据模式分割字符串,返回列表

```

re.split( , , a,s,d,asd )
[ a , s , d , asd ]      #返回列表

pat = re.compile( , )

```

```

pat.split( a,s,d,asd )
[ a , s , d , asd ]          #返回列表

re.split( [, ]+ , a , s ,d ,,,,asd )  #正则匹配: [, ]+, 后面说明
[ a , s , d , asd ]

re.split( [, ]+ , a , s ,d ,,,,asd ,maxsplit=2) # maxsplit 最多分割次数
[ a , s , d ,,,,asd ]

pat = re.compile( [, ]+ )          #正则匹配: [, ]+, 后面说明
pat.split( a , s ,d ,,,,asd ,maxsplit=2)      # maxsplit 最多分割次数
[ a , s , d ,,,,asd ]

```

findall(pattern,string): 列表形式返回匹配项

```

import re
print(re.findall( "a" , "ASDaDFGAa" ))
#[ a , a ]          #列表形式返回匹配到的字符串

pat =re.compile( "a" )
print(pat.findall( "ASDaDFGAa" ))
#[ a , a ]          #列表形式返回匹配到的字符串

pat = re.compile( "[A-Z]+" )      #正则匹配: [A-Z]+ 后面有说明
print(pat.findall( "ASDcDFGAa" ))
#[ ASD , DFGA ]          #找到匹配到的字符串

pat = re.compile( [A-Z] )
pat.findall( ASDcDFGAa )          #正则匹配: [A-Z]+ 后面有说明
[ A , S , D , D , F , G , A ]    #找到匹配到的字符串

pat = re.compile( [A-Za-z] )      #正则匹配: [A-Za-z]+ 匹配所有单词, 后面有说明
pat.findall( ASDcDFGAa )
[ A , S , D , c , D , F , G , A , a ]

```

sub(pat,repl,string): 用repl替换 pat匹配项
(留的是中间的, 因为中间在中心)

```

re.sub( a , A , abcasd )      #找到a用A替换, 后面见和group的配合使用
AbcAsd

pat = re.compile( a )
pat.sub( A , abcasd )
AbcAsd

pat=re.compile(r www.(.*)..{3} ) #正则表达式

#在Python的string前面加上'r', 是为了告诉编译器这个string是个raw string, 不要转译反斜杠。

```


#例如，在raw string中，是两个字符，和n，而不会转译为换行符。

#由于正则表达式和 会有冲突，因此，当一个字符串使用了正则表达式后，最好在前面加上 r 。

#与大多数编程语言相同，正则表达式里使用"\"作为转义字符，这就可能造成反斜杠困扰。

#假如你需要匹配文本中的字符"\"，那么使用编程语言表示的正则表达式里将需要4个反斜杠"\\\"：

#前两个和后两个分别用于在编程语言里转义成反斜杠，转换成两个反斜杠后再在正则表达式里转义成一个反斜杠。

#Python里的原生字符串很好地解决了这个问题，这个例子中的正则表达式可以使用r"\"表示。

#同样，匹配一个数字的"d"可以写成r"d"。

#有了原生字符串，你再也不用担心是不是漏写了反斜杠，写出来的表达式也更直观。

#不是说加了r就没有转译功能，好乱，就直接记住1句话：

#当一个字符串使用了正则表达式后，最好在前面加上 r ，这样你再也不用担心是不是漏写了反斜杠，写出来的表达式也更直观

```
pat.match( www.dxy.com ).group(1)
dxy
```

```
re.sub(r www.(.*)..{3} ,r , hello,www.dxy.com )
```

```
pat.sub(r , hello,www.dxy.com )
hello,dxy
```

r 1 是第一组的意思

#通过正则匹配找到符合规则的"www.dxy.com" ，取得 组1字符串 去替换 整个匹配。

```
pat=re.compile(r (w+) (w+) ) #正则表达式
s= hello world ! hello hz !
```

```
pat.findall( hello world ! hello hz ! )
[( hello , world ), ( hello , hz )]
```

```
pat.sub(r ,s) #通过正则得到组1(hello)，组2(world)，再通过sub去替换。
```

即组1替换组2，组2替换组1，调换位置。

```
world hello!hz hello!
```

escape(string)：对字符串里面的特殊字符串进行转义

```
re.escape( www.dxy.cn )
www\\.dxy\\.cn #转义
```

上面的函数中，只有match、search有group方法，其他的函数没有。

函数的方法

group：获取子模式(组)的匹配项

```

pat = re.compile(r www.(.)*(.) )      #用()表示1个组，2个组
m = pat.match( www.dxy.com )
m.group()                               #默认为0，表示匹配整个字符串
www.dxy.com

m.group(1)                             #返回给定组1匹配的子字符串
dxy

m.group(2)
com

```

start: 给定组匹配项的开始位置

```

m.start(2)                             #组2开始的索引
8

```

end: 给定组匹配项的结束位置

```

m.end(2) #组2结束的索引
11

```

span: 给定组匹配项的开始结束位置

```

m.span(2) #组2开始、结束的索引
(8, 11)

```

正则表达式

元字符

"." : 通配符,除换行符外的任意的1个字符

```

pat=re.compile( . )
pat.match( abc )
<_sre.SRE_Match object at 0xb72b6170>
pat.match( abc ).group()
a                                     #匹配到了首个字符
pat.search( abc ).group()
a
pat.match( ).group()                 #换行符匹配出错
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: NoneType object has no attribute group

```

"\" : 转义符

```

pat=re.compile( . )
pat.search( abc.efg ).group() #匹配到.
.
pat.findall( abc.efg )          #不用group, 返回列表
[ . ]

```

"[...]" : 字符集合, 匹配里面的任意一个元素

```

pat=re.compile( [abc] )
pat.match( axbycz ).group()
a
pat.search( axbycz ).group()
a
pat.findall( axbycz )
[ a , b , c ]

```

"d" : 数字

```

>>> pat=re.compile( d )
>>> pat.search( ax1by2cz3 ).group() #匹配到第一个数字:1, 返回
>>> 1

>>> pat.match( ax1by2cz3 ).group() #匹配不到(首个不是) 返回None, 报错, match匹配字符串头
>>> Traceback (most recent call last):
>>> File "<stdin>", line 1, in <module>
>>> AttributeError: NoneType object has no attribute group

>>> pat.findall( ax1by2cz3 )          #匹配所有的数字, 列表返回
>>> [ 1 , 2 , 3 ]

```

"D" : 非数字

```

>>> pat=re.compile( D )
>>> pat.match( ax1by2cz3 ).group()
>>> a
>>> pat.search( ax1by2cz3 ).group()
>>> a
>>> pat.findall( ax1by2cz3 )
>>> [ a , x , b , y , c , z ]
>>> 1
>>> 2
>>> 3
>>> 4
>>> 5
>>> 6
>>> 7
>>> "s" : 空白字符 \s
>>> \s

```

```
>>> 、空格

>>> pat=re.compile( s )
>>> pat.findall( ax1 by2 cz3 )
>>> [ , , , ]
>>> pat.search( ax1 by2 cz3 ).group()

>>> pat.match( ax1 by2 cz3 ).group()
```

“S”:非空白字符

```
>>> pat=re.compile( S )
>>> pat.search( ax1 by2 cz3 ).group()
>>> a
>>> pat.findall( ax1 by2 cz3 )
>>> [ a , x , 1 , b , y , 2 , c , z , 3 ]
```

“w”: 单个的 数字和字母, [A-Za-z0-9]

```
>>> pat=re.compile( w )
>>> pat.search( 1a2b3c ).group()
>>> 1
>>> pat.findall( 1a2b3c )
>>> [ 1 , a , 2 , b , 3 , c ]
>>> pat.match( 1a2b3c ).group()
```

“W”:非单词字符,除数字和字母外

```
>>> pat=re.compile( W )
>>> pat.findall( 1a2我b3c ) #python是用三字节表示一个汉字
>>> [ æ , ^ , ' ]
>>> pat.search( 1a2我b3c ).group()
>>> æ
```

数量词

“*”: 0次或多次

(乘0会变成0)

```
>>> pat = re.compile( [abc]* )
>>> pat.match( abcabcdefabc ).group()
>>> abcabc #2次
>>> pat.search( abcabcdefabc ).group()
>>> abcabc #2次
>>> pat.findall( abcabcdefabc )
>>> [ abcabc , , , abc , ] #2次和1次,因为有0次,所以匹配了
```

"+"：1次或多次

```
>>> pat = re.compile( [abc]+ )
>>> pat.match( abcdefabcabc ).group()
>>> abc
>>> pat.search( abcdefabcabc ).group()
>>> abc
>>> pat.findall( abcdefabcabc )
>>> [ abc , abcab ]
```

"?": 0次或1次, match,search 不会出现none, 会出现'' (因为0次也是符合的)
0次或1次不是指[xxx]这个集合, 而是其中的任何一个字符

```
>>> pat = re.compile( [abc]? )
>>> pat.match( defabc ).group()      #0次

>>> pat.match( abcdefabc ).group()
>>> a
>>> pat.search( defabc ).group()     #0次

>>> pat.findall( defabc )             #0次和1次
>>> [ , , , a , b , c , ]           #后面总再加个
```

"数量词?": 非贪婪模式: 只匹配最少的(尽可能少); 默认贪婪模式: 匹配最多的(尽可能多)

```
>>> pat = re.compile( [abc]+ )      #贪婪模式
>>> pat.match( abcdefabcabc ).group() #匹配尽可能多的: abc
>>> abc
>>> pat.match( bbabcdefabcabc ).group()
>>> bbabc
>>> pat.search( dbbabcdefabcabc ).group()
>>> bbabc
>>> pat.findall( abcdefabcabc )
>>> [ abc , abcab ]

>>> pat = re.compile( [abc]+? )     #非贪婪模式: +?
>>> pat.match( abcdefabcabc ).group() #匹配尽可能少的: a、b、c
>>> a
>>> pat.search( dbbabcdefabcabc ).group()
>>> b
>>> pat.findall( abcdefabcabc )
>>> [ a , b , c , a , b , c , a , b , c ]
```

"{m}": 匹配字符串出现m次

```
>>> pat = re.compile( [op]{2} )      #o或p出现2次
>>> pat.search( abcooapp ).group()    #匹配第一次出现的字符串,o比p先出现
>>> oo
>>> pat.findall( abcooapp )           #匹配出现的所有字符串,列表形式返回
>>> [ oo , pp ]
```

"{m,n}" : 匹配字符串出现m到n次

```
>>> pat = re.compile( [op]{2,4} )     #o或则p出现2到4次
>>> pat.match( pppabcooapp ).group()  #匹配开头
>>> ppp
>>> pat.search( pppabcooapp ).group()  #匹配第一次出现
>>> ppp
>>> pat.findall( pppabcooapp )         #匹配所有
>>> [ ppp , oo , pp ]
```

.group() #匹配第一次出现

边界

"^" : 匹配字符串开头或行头

```
>>> pat = re.compile( ^[abc] )        #开头是a、b、c中的任意一个
>>> pat.search( defabc ).group()
>>> pat.match( defabc ).group()        #均找不到
>>> pat.findall( defabc )
>>> []

>>> pat.search( adefabc ).group()
>>> a
>>> pat.match( adefabc ).group()       #开头是a、b、c中的任意一个
>>> a
>>> pat.findall( adefabc )
>>> [ a ]

>>> pat = re.compile( ^[abc]+ )        #开头是a、b、c中的任意一个的一次或则多次,贪婪:匹配多个
>>> pat.findall( cbadefab )
>>> [ cba ]
>>> pat = re.compile( r ^[abc]+? )     #开头是a、b、c中的任意一个的一次或则多次,非贪婪:匹配一个
>>> pat.findall( cbadefab )
>>> [ c ]
```

"\$" : 匹配字符串结尾或则行尾

```
>>> pat = re.compile( [abc]$ )
>>> pat.match( adefAbc ).group()    #match匹配的是字符串开头，所以查找$的时，总是返回None
>>> pat.search( adefAbc ).group()    #结尾是a、b、c中的任意一个
>>> c
>>> pat.findall( adefAbc )
>>> [ c ]
>>> pat = re.compile( [abc]+$ )
>>> pat.search( adefAbc ).group()    #结尾是a、b、c中的任意一个的一次或则多次，贪婪：匹配多个
>>> bc
>>> pat.findall( adefAbc )
>>> [ bc ]
```

“A”：匹配字符串开头

```
>>> pat = re.compile( A[abc]+ )
>>> pat.findall( cbadefab )
>>> [ cba ]
>>> pat.search( cbadefab ).group()
>>> cba
```

“Z”：匹配字符串结尾

```
>>> pat = re.compile( [abc]+z )
>>> pat.search( cbadefab ).group()
>>> ab
>>> pat.findall( cbadefab )
>>> [ ab ]
```

分组

(...): 分组匹配,从左到右,每遇到一个 (编号+1, 分组后面可加数量词

```
>>> pat=re.compile(r (a)w(c) )    #w:单个的数字或字母 [A-Za-z0-9]
>>> pat.match( abcdef ).group()
>>> abc
>>> pat=re.compile( (a)b(c) )    #分2组，匿名分组

>>> pat.match( abcdef ).group()    #默认返回匹配的字符串
>>> abc
>>> pat.match( abcdef ).group(1)    #取分组1，适用于search
>>> a
>>> pat.match( abcdef ).group(2)    #取分组2，适用于search
>>> c
>>> pat.match( abcdef ).groups()    #取所有分组，元组形式返回
>>> ( a , c )
```

: 引用编号为的分组匹配到的字符串

```
>>> pat=re.compile(r www.(.*)..{3} )
>>> pat.match( www.dxy.com ).group(1)
>>> dxy
```

"(?P...)"：在模式里面用()来表示分组（命名分组），适用于提取目标字符串中的某一些部位。

```
>>> pat=re.compile(r (?P<K>a)w(c) )      #分2组：命名分组+匿名分组
>>> pat.search( abcdef ).groups()        #取所有分组，元组形式返回
>>> ( a , c )
>>> pat.search( abcdef ).group(1)        #取分组1，适用于match
>>> a
>>> pat.search( abcdef ).group(2)        #取分组2，适用于match
>>> c
>>> pat.search( abcdef ).group()          #默认返回匹配的字符串
>>> abc
>>> pat.search( abcdef ).groupdict()      #命名分组可以返回一个字典【专有】，匿名分组也没有
>>> { K : a }
```

"(?P=name)"：引用别名为的分组匹配到的串

```
>>> pat=re.compile(r (?P<K>a)w(c)(?P=K) )      #(?P=K)引用分组1的值，就是a
>>> pat.search( abcdef ).group()              #匹配不到，因为完整 awca，模式的第4位是a
a
>>> Traceback (most recent call last):
>>> File "<stdin>", line 1, in <module>
>>> AttributeError: NoneType object has no attribute group

>>> pat.search( abcaedef ).group()            #匹配到，模式的第4位和组1一样，值是c
>>> abca
>>> pat.search( abcaedef ).groups()
>>> ( a , c )
>>> pat.search( abcaedef ).group(1)
>>> a
>>> pat.search( abcaedef ).group(2)
>>> c
```

""：引用分组编号匹配：


```
>>> pat=re.compile(r (?P<K>a)w(c)(?P=K) ) #引用分组2的值, 就是c
>>> pat.findall( Aabcacdef ) #匹配不到, 因为完整 awcac ,模式的第5位
是c
>>> []
>>> pat.findall( Aabcacdef ) #匹配到, 模式的第5位和组2一样, 值是c
>>> [( a , c )]
>>> pat.search( Aabcacdef ).groups()
>>> ( a , c )
>>> pat.search( Aabcacdef ).group()
>>> abcac
>>> pat.search( Aabcacdef ).group(1)
>>> a
>>> pat.search( Aabcacdef ).group(2)
>>> c
```

特殊构造

(?:...) (...)不分组版本,用于使用 | 或者后接数量词
 (?iLmsux) iLmsux的每个字符代表一个匹配模式,只能用在正则表达式的开头,可选多个
 (?#...) #号后的内容将作为注释
 (?=...) 之后的字符串内容需要匹配表达式才能成功匹配
 (?!...) 之后的字符串不匹配表达式才能成功
 (?(? (id/name) yes |no) 如果编号为id/名字为name的组匹配到字符串,则需要匹配yes,否则匹配no,no可以省略
 ...

“(?:...)” : ()里面有?:表示该()不是分组
 ```python
 >>> pat=re.compile(r a(?:bc) )
 >>> pat.findall( abc )
 >>> [ abc ]
 >>> pat.match( abc ).groups() #显示不出分组

“(?=...)”: 匹配...表达式, 返回。对后进行匹配, 总是对后面进行匹配

```
>>> pat=re.compile(r w(?:=d)) #匹配表达式d, 返回数字的前一位, w: 单词字符[A-Za-z0-9_]
>>> pat.findall(abc1 def1 xyz1)
>>> [c , f , z]
>>> pat.findall(zhoujy20130628hangzhou) #匹配数字的前一位, 列表返回
>>> [y , 2 , 0 , 1 , 3 , 0 , 6 , 2]
>>> pat=re.compile(r w+(?:=d))
>>> pat.findall(abc1,def1,xyz1) #匹配最末数字的前字符串, 列表返回
>>> [abc , def , xyz]
>>> pat.findall(abc21,def31,xyz41)
>>> [abc2 , def3 , xyz4]
>>> pat.findall(zhoujy20130628hangzhou)
>>> [zhoujy2013062]
>>> pat=re.compile(r [A-Za-z]+(?:=d)) #[A-Za-z], 匹配字母, 可以用其他的正则方法
>>> pat.findall(zhoujy20130628hangzhou123) #匹配后面带有数字的字符串, 列表返回
>>> [zhoujy , hangzhou]
>>> pat.findall(abc21,def31,xyz41)
```

```
>>> [abc , def , xyz]
```

“(?!...)” 不匹配...表达式, 返回。对后进行匹配

```
>>> pat=re.compile(r'[A-Za-z]+(?!\d)') # [A-Za-z], 匹配字母, 可以用其他的正则方法
>>> pat.findall('zhoujy20130628hangzhou123,12,binjiang310') # 匹配后面不是数字的字符串, 列表返回
>>> ['zhouj', 'hangzho', 'binjian']
>>> pat.findall('abc21,def31,xyz41')
>>> ['ab', 'de', 'xy']
```

"(?<=...)"：匹配...表达式，返回。对前进行匹配，总是对前面进行匹配

```
>>> pat=re.compile(r (?<=d)[A-Za-z]+) #匹配前面是数字的字母
>>> pat.findall(abc21,def31,xyz41)
>>> []
>>> pat.findall(1abc21,2def31,3xyz41)
>>> [abc , def , xyz]
>>> pat.findall(zhoujy20130628hangzhou123,12,binjiang310)
>>> [hangzhou]
```

"(?<!...)": 不匹配...表达式, 返回。对前进行匹配, 总是对前面进行匹配

```
>>> pat=re.compile(r '(?<!d)[A-Za-z]+') #匹配前面不是数字的字母
>>> pat.findall(abc21,def31,xyz41)
>>> [abc , def , xyz]
>>> pat.findall(zhoujy20130628hangzhou123,12,binjiang310)
>>> [zhoujy , angzhou , binjiang]
```

“(?(id/name) yes | no)”: 组是否匹配, 匹配返回

```
>>> pat=re.compile(r a(d)?bc?(1)d)) #no省略了，完整的是adbcd ==> a2bc3,总共5位，
第2位是可有可无的数字，第5为是数字
>>> pat.findall(abc9) #返回组1，但第2位（组1）没有，即返回了
>>> []
>>> pat.findall(a8bc9) #完整的模式，返回组1
>>> [8]
>>> pat.match(a8bc9).group()
>>> a8bc9
>>> pat.match(a8bc9).group(1)
>>> 8
>>> pat.findall(a8bc) #第5位不存在，则没有匹配到
>>> []
```

“(?!lmsux)”：这里就介绍下i参数：大小写区分匹配

```
>>> pat=re.compile(r abc)
>>> pat.findall(abc)
>>> [abc]
>>> pat.findall(ABC)
>>> []
>>> pat=re.compile(r (?i)abc) #(?i) 不区分大小写
>>> pat.findall(ABC)
>>> [ABC]
>>> pat.findall(abc)
>>> [abc]
>>> pat.findall(aBc)
>>> [aBc]
>>> pat.findall(aBC)
>>> [aBC]
>>> pat=re.compile(r abc ,re.I) #re.I 作为参数使用，推荐
>>> pat.findall(abc)
>>> [aBC]
>>> pat.findall(abc)
>>> [abc]
>>> pat.findall(ABC)
>>> [ABC]
```

### 3.3.3 提取数据

#### ◆使用正则表达式抽取

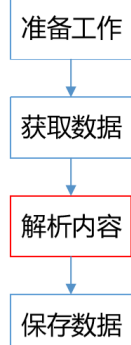
对需要的项进行处理，包括影片详情、图片链接和电影名等

```
影片详情链接
link=re.findall(findLink,item)[0]
data.append(link)#添加详情链接
imgSrc=re.findall(findImgSrc,item)[0]
data.append(imgSrc)#添加图片链接
titles=re.findall(findTitle,item)
#片名可能只有一个中文名，没有外国名
if(len(titles)==2):
 ctitle=titles[0]
 data.append(ctitle)#添加中文名
 ottitle=titles[1].replace("/","")#去掉无关符号
 data.append(ottitle)#添加外国片名
else:
 data.append(titles[0])#添加中文名
 data.append('')#留空
```

根据影片详情链接正则表达式抽取

根据图片链接正则表达式抽取

根据电影名正则表达式抽取



### ◆ 使用正则表达式抽取

对需要的项进行处理，包括评分、评价人数、电影说明、电影相关内容等。

```
rating=re.findall(findRating,item)[0]
data.append(rating)#添加评分
judgeNum=re.findall(findJudge,item)[0]
data.append(judgeNum)#添加评论人数
inq=re.findall(findInq,item)
#可能没有概况
if len(inq)!=0:
 inq=inq[0].replace("o","")#去掉句号
 data.append(inq)#添加概况
else:
 data.append('')#留空
bd=re.findall(findBd,item)[0]
bd=re.sub(remove,"",bd)
bd=re.sub('<br(\s+)?\s?>',' ',bd)#去掉

bd=re.sub('/'," ",bd)#替换/
data.append(bd.strip())
datalist.append(data)
```

根据评分正则表达式抽取

根据评价人数正则表达式抽取

根据电影说明正则表达式抽取

根据电影相关内容正则表达式抽取

data存储每一部电影抽取的结果

datalist存储所有电影解析结果

准备工作

获取数据

解析内容

保存数据

## 3.4 保存数据

### 3.4.1 Excel表存储

#### ◆ Excel表格存储

利用python库xlwt将抽取的数据datalist写入Excel表格

```
savepath='d:/home/asistudio/data/豆瓣电影Top250.xls'
saveData(datalist,savepath)
```

调用

#将相关数据写入excel中

```
def saveData(datalist,savepath):
 book=xlwt.Workbook(encoding='utf-8',style_compression=0)
 sheet=book.add_sheet('豆瓣电影Top250',cell_overwrite_ok=True)
 col=('电影详情链接','图片链接','影片中文名','影片外国名',
 '评分','评价数','概况','相关信息')
 for i in range(0,8):
 sheet.write(0,i,col[i])#列名
 for i in range(0,250):
 data=datalist[i]
 for j in range(0,8):
 sheet.write(i+1,j,data[j])#数据
 book.save(savepath)#保存
```

以utf-8编码创建一个Excel对象

创建一个Sheet表

往单元格写入内容

保存表格

准备工作

获取数据

解析内容

保存数据

## 补充：xlwt模块

简单使用xlwt

```
import xlwt #导入模块
workbook = xlwt.Workbook(encoding='utf-8') #创建工作book 对象
worksheet = workbook.add_sheet('sheet1') #创建工作表sheet
worksheet.write(0, 0, 'hello') #往表中写内容,第一各参数 行,第二个参数列,第三个参数内容
workbook.save('students.xls') #保存表为students.xls
```

课堂练习:

将九九乘法表显示在表格中, 每个单元格1个公式  
限定10分钟完成

```
workbook = xlwt.workbook(encoding='utf-8') #创建workbook 对象
worksheet = workbook.add_sheet('sheet1') #创建工作表sheet

for i in range(0,9):
 for j in range(0,i+1):
 worksheet.write(i, j, "%d * %d = %d"%(i+1,j+1,(i+1)*(j+1)))

worksheet.write(0, 0, 'hello') #往表中写内容,第一各参数 行,第二个参数列,第三个参数内容
workbook.save('students.xls') #保存表为students.xls
```

更多:

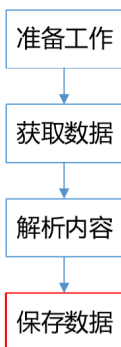
<https://www.jianshu.com/p/fc97dd7e822c>

<https://www.cnblogs.com/caesar-id/p/11802440.html>

#### ◆ Excel结果

电影详情链接、图片链接、影片中文名、影片外国名、评分、评价数等

| 电影详情链接                                                                                            | 图片链接                                                                                                                          | 影片中文名     | 影片外国名                                | 评分  | 评价数     | 概况              | 相关信息                                                              |
|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|-----------|--------------------------------------|-----|---------|-----------------|-------------------------------------------------------------------|
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 肖申克的救赎    | The Shawshank Redemption             | 9.6 | 1199754 | 希望让人自由          | 导演: 弗兰克·德拉邦特 Frank Darabont 主演: 蒂姆·罗宾斯 Tim Robbins                |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 霸王别姬      | The Farewell                         | 9.6 | 881629  | 风华绝代            | 导演: 陈凯歌 Kaige Chen 主演: 张国荣 Leslie Cheung 张丰毅 Fengyi Zhang         |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 这个杀手不太冷   | Léon                                 | 9.4 | 1104203 | 怪蜀黍和小萝莉         | 导演: 吕克·贝松 Luc Besson 主演: 让·雷诺 Jean Reno 娜塔莉·波特曼 Natalie Portman   |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 阿甘正传      | Forrest Gump                         | 9.4 | 945844  | 一部美国近现代史        | 导演: Robert Zemeckis 主演: Tom Hanks Robin Wright Penn Gary Sinise   |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 美丽人生      | La vita è bella                      | 9.5 | 552434  | 最美的谎言           | 导演: 罗伯托·贝尼尼 Roberto Benigni 主演: 罗伯托·贝尼尼 Roberto Benigni           |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 泰坦尼克号     | Titanic                              | 9.3 | 881502  | 失去的才是永恒         | 导演: 詹姆斯·卡梅隆 James Cameron 主演: 莱昂纳多·迪卡普里奥 Leonardo DiCaprio        |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 千与千寻      | 千と千尋の神隠し                             | 9.3 | 874327  | 最好的言语           | 导演: 宫崎骏 Hayao Miyazaki 主演: 柊瑠美 Rumi Hiragi 入野自由 Miyu              |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 辛德勒的名单    | Schindler's List                     | 9.5 | 496523  | 拯救一个人, 就是拯救一个民族 | 导演: 史蒂文·斯皮尔伯格 Steven Spielberg 主演: 连姆·尼森 Liam Neeson              |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 盗梦空间      | Inception                            | 9.3 | 963401  | 诺兰给了我们一个梦       | 导演: 克里斯托弗·诺兰 Christopher Nolan 主演: 莱昂纳多·迪卡普里奥 Leonardo DiCaprio   |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 机器人总动员    | WALL·E                               | 9.3 | 638846  | 小瓦力, 大人生        | 导演: 安德鲁·斯坦顿 Andrew Stanton 主演: 本·贝尔特 Ben Burtt 艾丽·古尔丁 Ali Hillman |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 忠犬八公的故事   | Hachi: A Dog's Tale                  | 9.3 | 622892  | 永远都不能忘记你        | 导演: 莱塞·霍尔斯道姆 Lasse Hallström 主演: 理查德·基尔 Richard Gere              |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 三傻大闹宝莱坞   | 3 Idiots                             | 9.2 | 862356  | 英俊版憨豆, 高智商      | 导演: 拉库马·希拉尼 Rajkumar Hirani 主演: 阿米尔·汗 Aamir Khan                  |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 海上钢琴师     | La leggenda del pianista sull'oceano | 9.2 | 720822  | 每个人都要走自己的路      | 导演: 朱塞佩·托纳多雷 Giuseppe Tornatore 主演: 蒂姆·罗斯 Tim Roth                |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 放牛班的春天    | Les choristes                        | 9.2 | 592836  | 天籁一般的童声         | 导演: 克里斯托夫·巴瑞斯特 Christophe Barratier 主演: 杰拉尔·朱诺 Géraud Larocque    |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 大话西游之大圣娶亲 | 西遊記大結局之仙履奇緣                          | 9.2 | 653474  | 一生所爱            | 导演: 刘镇伟 Jeffrey Lau 主演: 周星驰 Stephen Chow 吴孟达 Man Tat              |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 楚门的世界     | The Truman Show                      | 9.2 | 634151  | 如果再也见不到你        | 导演: 彼得·威尔 Peter Weir 主演: 金·凯瑞 Jim Carrey 劳拉·琳妮 Laura Linney       |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 教父        | The Godfather                        | 9.2 | 436949  | 千万不要忘记你         | 导演: 弗朗西斯·福特·科波拉 Francis Ford Coppola 主演: 马龙·白兰度 Marlon Brando     |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 龙猫        | となりのトトロ                              | 9.1 | 535448  | 人人心中都有个龙猫       | 导演: 宫崎骏 Hayao Miyazaki 主演: 日高法子 Noriko Hidaka 坂本千夏 Chika Sakamoto |
| <a href="https://movie.douban.com/subject/1292652/">https://movie.douban.com/subject/1292652/</a> | <a href="https://img3.douban.com/view/movie_review/p/1292652.jpg">https://img3.douban.com/view/movie_review/p/1292652.jpg</a> | 星际穿越      | Interstellar                         | 9.2 | 649891  | 爱是一种力量          | 导演: 克里斯托弗·诺兰 Christopher Nolan 主演: 马修·麦康纳 Matthew McConaughey     |



### 3.4.2 数据库存储

可以参考菜鸟教程:

Python和SQLite: <https://www.runoob.com/sqlite/sqlite-python.html>

SQLite 和SQL语句: <https://www.runoob.com/sqlite/sqlite-tutorial.html>

## 1.引入sqlite3库

```
import sqlite3
```

## 2.初始化数据库

demo:

```
import sqlite3

conn = sqlite3.connect('test.db')

print ("Opened database successfully")
```

创建数据表

```
import sqlite3

conn = sqlite3.connect('test.db')
print ("Opened database successfully")
c = conn.cursor()
c.execute(''''CREATE TABLE COMPANY
 (ID INT PRIMARY KEY NOT NULL,
 NAME TEXT NOT NULL,
 AGE INT NOT NULL,
 ADDRESS CHAR(50),
 SALARY REAL);''')
print ("Table created successfully")
conn.commit()
conn.close()
```

项目源码:

```
def init_db(dbpath):
 sql = "create table movie250 (id INTEGER primary key
 autoincrement,info_link text,
 pic_link text,cname varchar,ename varchar ,score numeric ,rated
 numeric ,introduction text,info text)" #创建数据表
 conn = sqlite3.connect(dbpath) #连接或创建数据库
 cursor = conn.cursor() #获取游标
 cursor.execute(sql) #执行SQL语句：创建数据表
 conn.commit() #事务提交：让操作生效
 cursor.close() #关闭游标
 conn.close() #关闭连接
```

## 3.数据库存储

插入数据：insert操作

```

import sqlite3

conn = sqlite3.connect('test.db')
c = conn.cursor()
print ("Opened database successfully")

c.execute("INSERT INTO COMPANY (ID,NAME,AGE,ADDRESS,SALARY) \
VALUES (1, 'Paul', 32, 'California', 20000.00);")

c.execute("INSERT INTO COMPANY (ID,NAME,AGE,ADDRESS,SALARY) \
VALUES (2, 'Allen', 25, 'Texas', 15000.00);")

c.execute("INSERT INTO COMPANY (ID,NAME,AGE,ADDRESS,SALARY) \
VALUES (3, 'Teddy', 23, 'Norway', 20000.00);")

c.execute("INSERT INTO COMPANY (ID,NAME,AGE,ADDRESS,SALARY) \
VALUES (4, 'Mark', 25, 'Rich-Mond ', 65000.00);")

conn.commit()
print ("Records created successfully")
conn.close()

```

项目源码:

```

#datalist是封装好的数据，dbpath是数据库文件存放的全路径
def saveData2(datalist,dbpath):
 init_db(dbpath) # 创建数据表
 con = sqlite3.connect(dbpath) #连接数据库
 cur = con.cursor() #获取游标
 for data in datalist: #对每行数据进行操作
 for index in range(len(data)): #index是每行数据的下标
 data[index] = ("\"" + data[index] + "\"") #对每个数据添加前后的双引号，\是转义字符

 sql = 'INSERT INTO
movie250(info_link,pic_link,cname,ename,score,rate,introduction,info) VALUES
(%s)' % ",".join(data) #拼接建表语句，连接data列表中的每一项，使用逗号分隔

 cursor.execute(sql) #执行SQL语句：创建数据表
 conn.commit() #事务提交：让操作生效

 cur.close() #关闭游标
 conn.close() #关闭连接

```

#### 4.数据库查询（后面项目用的上）

select操作:

下面的 Python 程序显示了如何从前面创建的 COMPANY 表中获取并显示记录:

```
import sqlite3

conn = sqlite3.connect('test.db')
c = conn.cursor()
print ("Opened database successfully")

cursor = c.execute("SELECT id, name, address, salary from COMPANY")
for row in cursor:
 print ("ID = ", row[0])
 print ("NAME = ", row[1])
 print ("ADDRESS = ", row[2])
 print ("SALARY = ", row[3], "\n")

print("Operation done successfully")
conn.close()
```

5.修改操作: update (略)

6.删除操作: delete (略)

### 3.4.3 数据展示 (了解)

#### ◆ 读取数据并展示

```
import pandas as pd
df = pd.read_excel("/home/aistudio/data/豆瓣电影Top250.xls")
print(df.head())
```

```

 电影详情链接 \
0 https://movie.douban.com/subject/1282052/
1 https://movie.douban.com/subject/1291545/
2 https://movie.douban.com/subject/1295644/
3 https://movie.douban.com/subject/1292720/
4 https://movie.douban.com/subject/1292003/

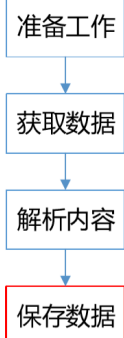
 图片链接 影片中文名 \
0 https://img3.doubanio.com/view/photo/s_ratio_p... 肖申克的救赎
1 https://img3.doubanio.com/view/photo/s_ratio_p... 霸王别姬
2 https://img3.doubanio.com/view/photo/s_ratio_p... 这个杀手不太冷
3 https://img1.doubanio.com/view/photo/s_ratio_p... 阿甘正传
4 https://img3.doubanio.com/view/photo/s_ratio_p... 美丽人生

 影片外国名 评分 评价数 概况 \
0 The Shawshank Redemption 9.5 1225578 希望让人自由
1 9.5 901901 风华绝代
2 Lifen 9.4 1127321 怪蜀黍和小萝莉不得不说的故事
3 Forrest Gump 9.4 965497 一部美国近现代史
4 La vita è bella 9.5 564509 最美的谎言
```

#### ◆ 查看数据基本信息

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 250 entries, 0 to 249
Data columns (total 8 columns):
 电影详情链接 250 non-null object
 图片链接 250 non-null object
 影片中文名 250 non-null object
 影片外国名 250 non-null object
 评分 250 non-null float64
 评价数 250 non-null int64
 概况 250 non-null object
 相关信息 250 non-null object
dtypes: float64(1), int64(1), object(6)
memory usage: 15.7+ KB
```





## ◆ 评分和排名的关系

```
import matplotlib.pyplot as plt
import matplotlib
#配置中文字体和修改字体大小
matplotlib.rcParams['font.family']='SimHei'
matplotlib.rcParams['font.size']=20
plt.figure(figsize=(20,5))
plt.subplot(1,2,1)
plt.scatter(df['评分'],range(1,251))
plt.xlabel('评分')
plt.ylabel('排名')
#修改y轴为倒序
plt.gca().invert_yaxis()
#集中趋势的直方图
plt.subplot(1,2,2)
plt.hist(df['评分'],bins=15)
```

